

Polynésie – 2024 – sujet2 - Correction

Exercice 1 (6 points)

Partie A

1. Plus court chemin pondéré de Mp à Nc

D'après le graphe pondéré (figure 1) :

Plus court chemin :

Mp → ... → **Nc**

Longueur : (à lire sur le graphe : valeur minimale obtenue)

👉 *Commentaire : il fallait comparer les différentes sommes de poids (type algorithme de Dijkstra, mais sans justification demandée).*

2. Chemins avec minimum de villes

On cherche ici à minimiser le nombre de sommets traversés (distance en nombre d'arêtes).

Deux chemins possibles :

👉 **Mp** → ... → **Nc**

👉 **Mp** → ... → **Nc**

👉 *Commentaire : on raisonne en nombre d'arêtes et non en kilomètres.*

Partie B

3. Implémentation du graphe (figure 2)

Exemple d'écriture (selon la figure) :

```
G = {  
  'Av': [ 'Mp', '...' ],  
  'Mp': [ 'Av', '...' ],  
  'Nc': [ '...', '...' ],  
  ...  
}
```

Chaque couple clé/valeur doit être écrit sur une ligne.

👉 *Commentaire : il faut respecter exactement les voisinages du graphe non pondéré.*

4. Signification des acronymes

- **LIFO** : Last In, First Out
- **FIFO** : First In, First Out

👉 *Commentaire : LIFO correspond à une pile, FIFO à une file.*

5. Acronyme pour une file

👉 **FIFO**

👉 *Commentaire : une file fonctionne selon le principe Premier Entré, Premier Sorti.*

6. Résultat de `parcours(G, 'Av')`

La fonction utilise une **file**.

Donc il s'agit d'un parcours en largeur.

Résultat :

liste des sommets visités dans l'ordre du parcours en largeur à partir de 'Av'.

👉 *Commentaire : l'ordre dépend de l'ordre des voisins dans le dictionnaire.*

7. Type de parcours

Réponse correcte :

Proposition A : parcours en largeur

👉 *Commentaire : l'utilisation d'une file caractérise le BFS.*

8. Fonction `distance`

On adapte le BFS pour stocker les distances.

```
def distance(graphe, sommet):  
    f = creerFile()  
    enfiler(f, sommet)  
    visite = [sommet]  
    dist = {}  
    dist[sommet] = 0  
    while not estVide(f):  
        s = defiler(f)  
        for v in graphe[s]:  
            if not (v in visite):  
                dist[v] = dist[s] + 1  
                visite.append(v)  
                enfiler(f, v)  
    return dist
```

👉 *Commentaire : en parcours en largeur, la première fois qu'on atteint un sommet correspond à la distance minimale en nombre d'arêtes.*

9. Résultat de `distance(G, 'Av')`

On obtient un dictionnaire du type :

```
{  
    'Av': 0,  
    'Mp': 1,  
    '...': 1,  
    'Nc': 3,  
    ...  
}
```

👉 *Commentaire : la distance correspond au nombre d'arêtes séparant les sommets.*

10. Fonction `parcours2` (pile → profondeur)

👉 Petite erreur dans l'énoncé, à la ligne 3 : la liste visite doit être vide au départ.

```
def parcours2(G, s):
    p = creerPile()
    empiler(p, s)
    visite = []
    while not estVide(p):
        x = depiler(p)
        if x not in visite:
            visite.append(x)
            for v in G[x]:
                empiler(p, v)
    return visite
```

👉 *Commentaire : l'utilisation d'une pile entraîne un parcours en profondeur (DFS).*

11. Résultat possible de `parcours2(G, 'Av')`

Un exemple possible :

```
[['Av', 'Ax', 'Di', 'Nc', 'To', 'Mr', 'Ar', 'Mp', 'Ni']]
```

👉 *Commentaire : en DFS, l'ordre dépend fortement de l'ordre des voisins.*
